Information Technology Course
Module Software Engineering
by Dr. Damir Dobric / Andreas Pech

ML-25-26-02-AI-Document-Classification-Agent-PlugIn

Sneha Mallepalli (1630732)
sneha.mallepalli@stud.fra-uas.de

Sushma Prabhugowda Patil (1628777)
sushma.prabhugowda-patil@stud.fra-uas.de

Abstract - The increased use of digital documents has created challenges in efficiently organizing, classifying, and retrieving information across diverse formats. Conventional approaches, such as keyword-based and rule-based methods, often fail to capture semantic meaning and contextual relationships within unstructured text, leading to limited classification accuracy. This paper presents an artificial intelligence-based document classification and retrieval system that leverages semantic embeddings to enhance document understanding and search performance. The system processes documents in multiple formats, including text, portable document format files, word documents, and images, using a document parsing framework with optical character recognition support. Extracted text is transformed into high-dimensional vector representations and stored in a structured database along with class-level semantic representations. The system uses a two-stage retrieval to find the best result. First, it compares the user query with different categories to find the most relevant one. Then, it compares the query with documents inside that category to find the best matching document. This approach improves accuracy and shows that embedding-based methods work better than traditional techniques.

Keywords - Document Classification, Semantic Search, Text Embeddings, Cosine Similarity, Natural Language Processing, Document Retrieval, Optical Character Recognition.

I. INTRODUCTION

The rapid growth of digital information has increased the need for efficient document management systems capable of organizing and retrieving information across diverse formats. Organizations generate large volumes of documents such as reports, articles, and scanned images, making manual classification inefficient and difficult to scale. Machine learning-based classification approaches have been widely used to automate this process [1], while traditional keyword-based retrieval methods often fail to capture semantic meaning, resulting in limited accuracy [6].

Recent advancements in natural language processing have introduced embedding-based techniques that represent text as high-dimensional vectors, enabling semantic comparison between documents. Early models such as Word2Vec demonstrated the ability to learn meaningful word relationships from large datasets [2], while transformer-based architectures improved contextual understanding through attention mechanisms [3].

Modern approaches such as Sentence-BERT enable efficient sentence-level similarity, making them suitable for document retrieval tasks [4]. Further improvements using contrastive learning methods such as SimCSE enhance the quality of semantic representations [5]. These similarity-based approaches are conceptually related to nearest neighbor methods used in classification tasks [7].

In addition, document processing technologies such as optical character recognition (OCR) allow text extraction from images and scanned files, enabling support for multiple document formats [8].

Based on these advancements, this paper presents an AI-based document classification and retrieval system using semantic embeddings and a two-stage similarity approach. The system processes documents, generates embeddings using the OpenAI model [9], and stores them in a structured database. During query processing, the system first identifies the most relevant category and then retrieves the best matching document, improving semantic understanding compared to traditional methods.

II. LITERATURE SURVEY

A. Limitations of Traditional Classification Methods

Early document classification systems used rule-based approaches and keyword matching, which required manually designed features. Ikonomakis et al. reviewed machine learning methods for text classification and showed that algorithms such as Naïve Bayes, k-Nearest Neighbors, and Support Vector Machines can achieve high accuracy when features are carefully created from labeled data [1].

However, these methods rely on term frequency representations that ignore word order and context, making them less effective for complex queries. They are sensitive

to changes in vocabulary, struggle with synonyms, and require retraining when new document categories are added

B. Distributed Word and Sentence Representations

Word2Vec represents words as vectors in a continuous high-dimensional space, where words with similar meanings are located close to each other [2]. The CBOW and Skip-gram models capture semantic relationships between words through vector operations, showing that semantic similarity can be represented as geometric closeness in embedding space. This idea forms the basis of modern embedding-based document classification, where similar documents are expected to cluster together.

The Transformer architecture uses self-attention mechanisms instead of recurrent or convolutional layers to process text [3]. Its ability to capture long-range dependencies through multi-head attention has become the foundation of modern language models and embedding systems. Embedding models such as those provided by OpenAI follow these principles to encode contextual meaning into dense vector representations.

Sentence-BERT (SBERT) generates meaningful sentence embeddings by fine-tuning BERT using siamese and triplet network structures [4]. These embeddings are well-suited for similarity comparison tasks, highlighting the importance of embedding quality in semantic retrieval systems.

C. Contrastive Learning and Embedding Quality

SimCSE highlights the effectiveness of sentence embeddings for similarity-based tasks by using contrastive learning techniques [5]. High-quality embeddings require both alignment, which brings similar sentences closer together, and uniformity, which ensures that embeddings are well distributed across the vector space. These properties improve the ability of embeddings to represent semantic relationships.

As a result, embedding models trained with contrastive objectives are well-suited for tasks involving semantic similarity and retrieval-based systems.

D. Cosine Similarity in Information Retrieval

Cosine similarity is a standard metric used in vector-space information retrieval to measure similarity between two vectors [6]. It computes the cosine of the angle between vectors, providing a scale-invariant measure that is not affected by differences in document length.

This makes it well-suited for comparing document embeddings with varying sizes and content densities. As a result, it is widely used in semantic retrieval systems to rank documents based on relevance.

E. Similarity-Based Classification Approaches

Nearest-neighbor classification methods use distance-based approaches to assign class labels based on similarity in feature space [7]. These methods are closely related to embedding-based classification systems, where similarity in vector space determines relevance.

In such systems, class-level representations like centroids can approximate category semantics, allowing efficient comparison between queries and groups of documents.

F. Optical Character Recognition for Document Digitization

The Tesseract OCR engine enables text extraction from scanned and image-based documents using adaptive classification techniques [8]. Modern versions use LSTM-based recognition, improving performance on complex and degraded inputs.

In document processing systems, OCR allows text to be extracted from image formats such as PNG and JPEG, enabling them to be processed alongside text-based formats like PDF and DOCX. However, the quality of extracted text can vary depending on the input conditions.

G. Research Gap and Motivation

Despite these advancements, existing approaches either focus on document-level retrieval or require computationally expensive comparisons across all documents. There is limited work on efficient classification frameworks that combine class-level abstraction with document-level precision in a single pipeline.

To address this, the proposed system uses a two-stage retrieval mechanism. It first uses class centroid representations for efficient category selection, followed by document-level similarity ranking to identify the most relevant document.

III. METHODOLOGY

The proposed system implements a document classification architecture organized as a structured pipeline with a conditional execution path. The overall workflow is illustrated in *Fig. 1*. The pipeline consists of two phases: a document processing phase and a query classification phase.

A. System Configuration

The pipeline begins by loading the system configuration, which includes the API credentials for the

embedding model, the database connection parameters, and the document directory path. This configuration drives all subsequent components without requiring manual intervention at runtime.

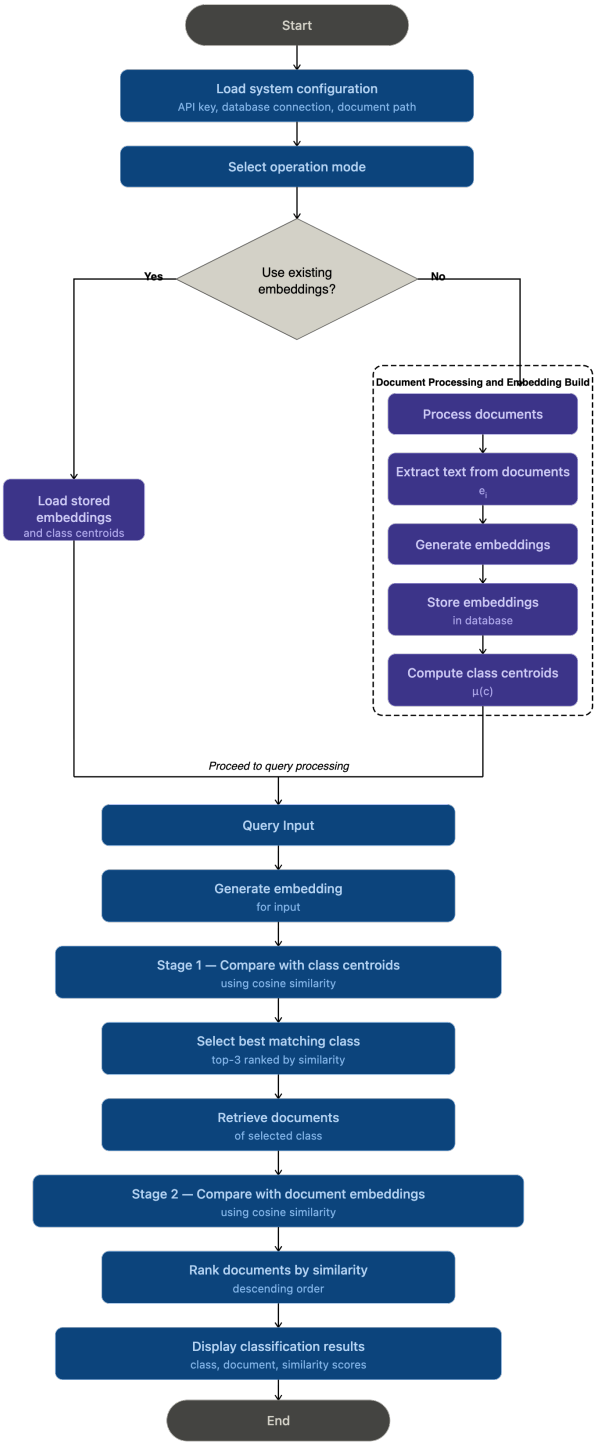

Fig. 1 Graphical representation of the AI Document Classification and Agent methodology.

B. Mode Selection

Following configuration, the system presents an operation mode selector. The user selects one of two primary paths: the fast path, which loads stored embeddings and class centroids directly from the database; or the build path, which triggers the full document processing pipeline. This decision branch allows the system to skip embedding generation when embeddings have already been computed.

C. Document Processing and Embedding Build

When the build path is selected, the system processes all documents in the dataset. The system supports multiple document formats including plain text, PDF, Word documents, and image-based files processed through OCR. Text is extracted from each document regardless of format. The extracted text is passed to the embedding model, which produces a dense vector representation of the document content. Each embedding is stored in the database for subsequent retrieval.

After all documents within a class are processed, the class centroid is computed as the element-wise arithmetic mean of all document embedding vectors in that class.

Each document is represented as a high-dimensional embedding vector (e.g., a list of numerical values capturing its semantic meaning). To compute the centroid, the system averages each corresponding dimension across all document embeddings in the class.

For example, if a class contains three document embeddings:

$e_1 = [x_1, x_2, x_3]$,
$e_2 = [y_1, y_2, y_3]$,
$e_3 = [z_1, z_2, z_3]$,

then the centroid μ(c) is computed as:

$$\mu(c) = [(x_1 + y_1 + z_1)/3, (x_2 + y_2 + z_2)/3, (x_3 + y_3 + z_3)/3]$$

Formally, given a class C containing n documents with embeddings $e_1, e_2, ..., e_n$, the centroid μ(c) is defined as:

$$\mu(c) = (1/n) \times \Sigma\, e_i, \text{ for } i = 1 \text{ to } n$$

where each e_i is a vector and the resulting centroid is also a vector of the same dimension. The centroid represents the overall semantic profile of the class and is used during Stage 1 retrieval to identify the most relevant class for a given query.

The generated embeddings and computed class centroids are stored in an SQL database, enabling reuse during subsequent query processing.

D. Query Input

Once embeddings and centroids are available, either loaded from the database or freshly computed, the

system proceeds to query processing. The user submits a natural language query or an external document for classification.

E. Query Embedding Generation

The input query is converted into a dense vector representation using the same embedding model employed during the document processing phase. Using the same embedding model for both documents and queries ensures that all vectors exist in the same semantic vector space, enabling consistent similarity comparisons throughout both retrieval stages.

F. Two-Stage Retrieval Design

The system adopts a two-stage retrieval strategy. In the first stage, the query embedding is compared with class centroid vectors to identify the most relevant category. In the second stage, the query is compared with individual document embeddings within the selected class to identify the most relevant document. This design reduces the number of similarity comparisons compared to a flat comparison across all documents.

G. Stage 1 — Comparison with Class Centroids

The query embedding is compared against all stored class centroid vectors using cosine similarity. Cosine similarity between two vectors A and B is defined as:

$$\text{Cos}(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

This metric is scale-invariant and robust to variations in document length, making it appropriate for comparing embeddings of heterogeneous documents [6]. The similarity computation is performed once per stored class centroid. The top three classes are ranked by their similarity scores.

H. Class Selection

The class with the highest cosine similarity score is selected as the predicted category. This follows a prototype-based classification approach, where each class centroid serves as a representative vector for its category. The ranked list of top classes provides a similarity-based indication of relevance for each candidate.

I. Retrieve Documents of Selected Class

All document embeddings belonging to the selected class are retrieved from the database. This step limits the comparison space to only the documents within the predicted category, reducing the number of similarity computations in Stage 2 compared to a flat search across all documents.

J. Stage 2 — Comparison with Document Embeddings

The query embedding is compared against each individual document embedding within the selected class using cosine similarity. The same metric applied in Stage 1 is used here, ensuring consistency across both retrieval stages. Data representation consistency between query and document embeddings, guaranteed by using the same model for all encoding, ensures that similarity scores are meaningful and comparable across both stages.

K. Ranking and Output

All documents within the selected class are ranked in descending order of their cosine similarity score with the query. The document with the highest similarity score is selected as the best matching result. The system displays: the top 3 predicted classes with their similarity scores, the predicted class name and its similarity score, and the best matching document with its corresponding similarity score.

IV. IMPLEMENTATION

A. Technology Stack

The system is implemented using modern software engineering tools and frameworks to support scalability, modularity, and efficient processing. The overall technology stack used in the implementation is summarized in Table I.

Table I : Technology Stack

Component	Technology
Language & Runtime	C# / .NET 9.0
Database	MS SQL Server
ORM	Entity Framework Core
Embedding Model	OpenAI Embeddings API
Document Parser	Daenet.DocumentParser
OCR Engine	Tesseract OCR
Similarity	Cosine Similarity
Console UI	Spectre.Console
Testing Framework	MSTest

The overall system architecture consists of two main components: a **Classifier module** and an **Agent module**, each responsible for different stages of the document classification and retrieval pipeline. The classifier module handles document ingestion, preprocessing, embedding generation, and storage, while the agent module performs query processing and semantic retrieval using similarity-based techniques.

B. Classifier Module

The classifier module is responsible for processing input documents and preparing them for semantic retrieval.

The overall workflow of this module is illustrated in **Error! Reference source not found.**.

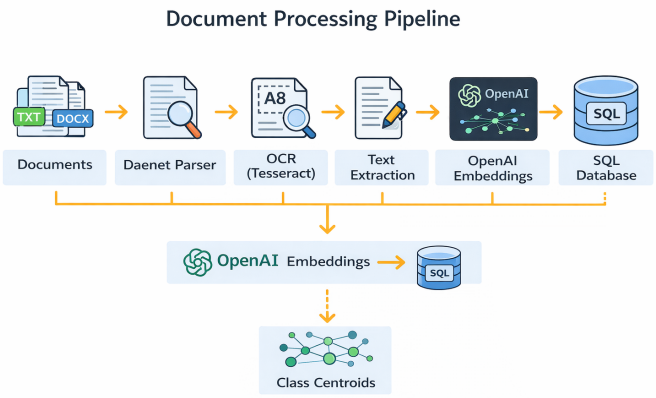

Fig. 2 Document processing pipeline showing document parsing, OCR-based text extraction, embedding generation, storage in SQL database, and centroid computation

1. *Document Processing and OCR*

The process begins with document ingestion, where documents in multiple formats such as text, PDF, DOCX, and images are provided as input. The system uses the external library Daenet.DocumentParser, integrated through the ExternalDocumentPreprocessor, to extract content from these documents.

For image-based inputs, optical character recognition is applied using Tesseract through the parser configuration. This allows the system to extract text from scanned documents and images, ensuring that all document types can be processed uniformly.

2. *Text Extraction and Preparation*

After parsing and OCR, the extracted content is converted into plain text. This step ensures that both structured and unstructured documents are represented in a consistent format before further processing.

3. *Embedding Generation*

The processed text is converted into semantic embeddings using the IEmbeddingGenerator interface. The implementation utilizes the OpenAI embedding model to generate high-dimensional vector representations of the text. These embeddings capture the semantic meaning of documents and enable similarity-based comparison. Each document is associated with an embedding stored along with its metadata.

4. *Database Storage*

The system uses a structured SQL database to persist document data and embeddings. The persistence layer is implemented using the IEmbeddingRepository interface and its concrete implementation SqlEmbeddingRepository.

The database stores:

- DocumentClasses
- Documents
- DocumentEmbeddings
- ClassCentroids

This design ensures efficient retrieval and supports dynamic updates when new documents are added.

5. *Centroid Computation*

Centroid computation is implemented using the CentroidCalculator component, which computes the average embedding vector for each document class. The DocumentClassificationRunner handles this process by triggering centroid recomputation after document embeddings are generated. Class centroids represent the semantic center of each category and are used during query processing to efficiently identify the most relevant class.

C. *Agent Module*

The agent module is responsible for processing user queries and retrieving the most relevant document. The overall workflow is illustrated in Fig. 2.

.

1. *Query Embedding Generation*

When a user submits a query, it is converted into an embedding using the AgentIntentEmbeddingService. This service uses the same OpenAI embedding model as the classifier, ensuring that queries and documents exist in the same semantic vector space for meaningful comparison.

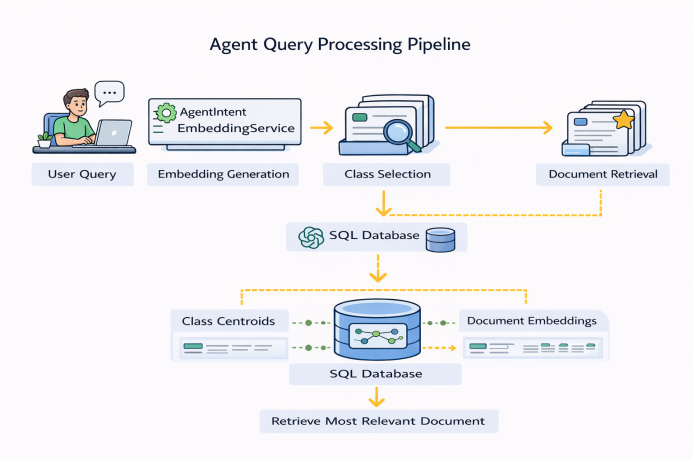

Fig. 2 Agent query processing pipeline showing query embedding generation, class selection using centroid similarity, and document retrieval from the SQL database

2. *Class Selection*

The system compares the query embedding with class centroid embeddings using the AgentSimilarityMatcher. Cosine similarity is used to compute similarity scores, and classes are ranked accordingly.

This step reduces the search space by selecting only the most relevant class.

3. *Document Retrieval*

Once a class is selected, the system retrieves document embeddings belonging to that class from the database. This targeted retrieval improves efficiency by avoiding unnecessary comparisons.

4. *Document-Level Ranking*

The query embedding is compared with document embeddings using cosine similarity to identify the most relevant document. This step performs fine-grained ranking within the selected class.

5. *Result Handling*

The result is formatted using the AgentResultFormatter, which presents the best matching document along with similarity scores. The system also handles edge cases such as empty results or low similarity matches.

D. End-to-End Retrieval Strategy

The system implements a two-stage retrieval strategy:

1. Class-level filtering using centroid similarity.
2. Document-level ranking using embedding similarity.

This design balances efficiency and accuracy by minimizing unnecessary comparisons while maintaining precise results.

E. Design Considerations

The system is designed using dependency injection, enabling loose coupling between components. This improves maintainability and allows individual modules such as preprocessing, embedding generation, and retrieval to be tested independently.

F. System Features and Enhancements

In addition to the core requirements, the system incorporates several enhancements to improve flexibility, efficiency, and usability.

1. *Embedding Regeneration*

The system provides multiple options for regenerating embeddings, as shown in **Error! Reference source not found.**. Users can:

- Rebuild all embeddings
- Rebuild embeddings for a specific class
- Rebuild embeddings for an individual document

These options are available through the agent startup menu and allow selective recomputation. This design avoids unnecessary processing and ensures that updates to documents can be reflected without rebuilding the entire dataset.

```
Agent Mode
-----------
Type your query and press Enter.
Type 'menu' to return to main menu.
Type 'exit' or 'E' to quit.

[Agent] Reloading centroids and embeddings...
[Agent] Ready for queries.
--------------------------------
===== Agent Startup Options =====
1. Use existing embeddings
2. Rebuild ALL embeddings
3. Rebuild embeddings for a CLASS
4. Rebuild embedding for a FILE
5. Upload document and classify
Select option: 1
[Agent] Using existing embeddings.

>> how to go to Paris?

Top matching classes:
1. Travel (score: 0.2626)
2. SpaceExploration (score: 0.1586)
3. ClimateChange (score: 0.1406)

Predicted class: Travel
Class similarity score: 0.2626
Best matching document: Popular_Tourist_Destinations.pdf
Document similarity score: 0.2771
```

Fig. 3 Sample agent execution showing embedding regeneration options and document classification output.

2. *Incremental Document Processing*

The system supports uploading new documents dynamically for classification. As illustrated in **Error! Reference source not found.**, users can select the option to upload a document, which is then processed through parsing, text extraction, and embedding generation.

Only the new document is processed, while existing embeddings remain unchanged. This incremental approach improves scalability and enables efficient handling of large document collections.

3. *Duplicate Handling and Optimization*

Before generating embeddings, the system checks whether an embedding already exists for a document. If an embedding is present, the system skips regeneration to avoid redundant computation. This optimization improves performance and ensures efficient use of resources.

4. *Flexible Input Structure*

The system supports a flexible directory-based input structure, where each folder represents a document class and can contain multiple documents. This design allows the system to adapt to different use cases without requiring changes to the core implementation

5. *Interactive Console-Based Workflow*

The system provides a structured command-line interface with multiple operational options, including embedding regeneration and document classification. The

menu-driven approach simplifies user interaction while maintaining flexibility for advanced operations.

V. EXPERIMENTAL RESULTS

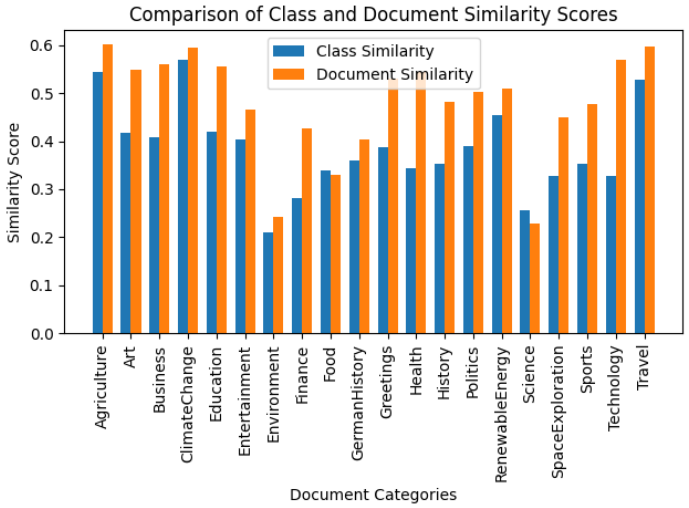

Fig. 4 Comparison of class-level and document-level similarity scores across document categories.

A. Similarity Score Analysis

To evaluate the semantic performance of the system, similarity scores were analysed at both the class level and document level. Fig. 4 shows the comparison of similarity scores across different document categories. It can be observed that document-level similarity scores are consistently higher than class-level similarity scores. This confirms the effectiveness of the two-stage retrieval mechanism, where class centroids are first used to identify the most relevant category, followed by document-level comparison to retrieve the best matching document.

Fig. 5 presents the trend of similarity scores across all categories. The results demonstrate stable performance across diverse query types. Higher similarity values are observed for domain-specific queries, while more general queries result in relatively lower scores.

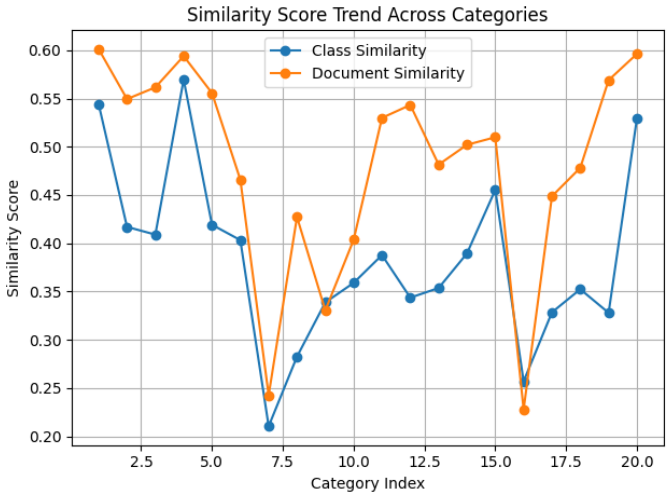

Fig. 5 Trend of similarity scores across different document categories

B. Accuracy Evaluation

The classification accuracy was calculated by comparing the number of correct predictions with the total number of evaluated queries. Out of approximately 120 queries, around 85–90 were correctly classified, resulting in an overall accuracy of approximately 70–75%. The evaluation includes both meaningful queries and noisy inputs to reflect realistic usage scenarios.

The system performs reliably for context-rich queries, where semantic meaning is clearly defined. However, accuracy decreases for short or ambiguous inputs.

C. Category-wise Performance Analysis

The system shows varying performance across different document categories. Categories such as ClimateChange, Travel, Agriculture, and RenewableEnergy demonstrate high similarity scores and accurate classification results. These categories benefit from well-defined semantic boundaries and consistent document content.

In contrast, categories such as Environment and Science exhibit relatively lower similarity scores. This is mainly due to semantic overlap between related categories and the presence of diverse topics within a single class. For example, Environment shares contextual similarity with ClimateChange and Agriculture, which can reduce classification confidence.

D. Error Analysis

Error analysis indicates that incorrect predictions primarily occur in the following scenarios:

- Short or incomplete queries (e.g., single-word inputs)
- Ambiguous queries with multiple possible meanings
- Noisy inputs such as random strings or numerical values

In such cases, the system may produce incorrect classifications due to insufficient semantic information. Additionally, overlapping categories contribute to misclassification when the query context is not clearly distinguishable.

E. Impact of Similarity Threshold

To improve system reliability, a minimum similarity threshold of 0.15 is introduced. If the similarity score falls below this threshold, the system returns *"No relevant document found (below threshold)"* instead of forcing a prediction.

This enhancement significantly reduces false positives and improves the robustness of the system, particularly for noisy or low-confidence inputs. As a result,

the system provides more meaningful and reliable outputs in real-world scenarios.

VI. UNIT TESTS

To validate the correctness and reliability of the implemented document classification system, a series of unit tests were developed using the MSTest framework. The tests focus on verifying the behaviour of critical components within both the classifier module and the agent module, including centroid calculation, document classification processing, similarity ranking, embedding generation, and result formatting. Mock dependencies were created using the Moq framework to isolate individual components and simulate external services such as embedding generation and database persistence.

The results of the test execution are shown in Fig. 6, where implemented tests passed successfully.

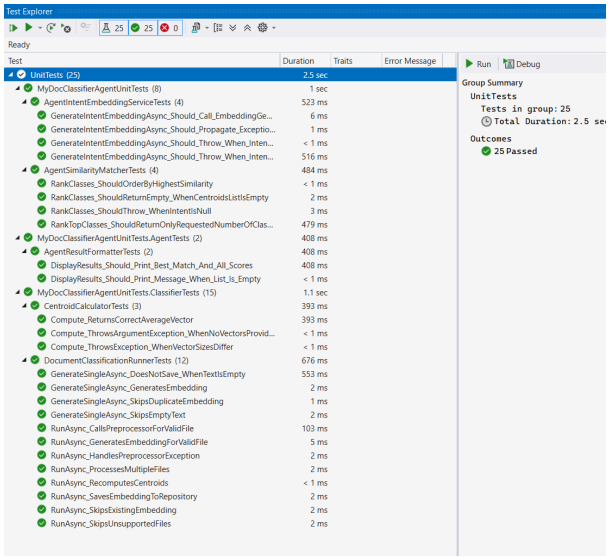

Fig. 6 Execution results of the unit test suite showing successful completion of all tests

A. Centroid Calculation Tests

Objective: This test evaluates the functionality of the centroid calculation component, ensuring that the system correctly computes the average embedding vector for a given set of document embeddings.

Test Scenario: A set of embedding vectors is provided as input to the centroid calculator. The system computes the centroid by calculating the element-wise average of the vectors. Additional scenarios include providing an empty list of vectors and vectors with inconsistent dimensions to verify the method's robustness against invalid inputs.

Outcome Verification: The test verifies that the returned centroid vector contains the correct average values for each dimension. It also confirms that the method throws an ArgumentException when no vectors are provided or when vector dimensions differ.

Benefit: These tests ensure that the centroid computation used for class representation is mathematically correct and robust against invalid input conditions.

B. Document Classification Runner Tests

Objective: These tests examine the behaviour of the document classification pipeline, ensuring that documents are processed correctly and embeddings are generated and stored appropriately.

Test Scenario: The tests simulate different execution scenarios using mocked dependencies. These include processing valid documents, skipping unsupported file types, avoiding duplicate embeddings when embeddings already exist, and processing multiple documents within a directory. The tests also simulate exceptions in the document preprocessing stage.

Outcome Verification: The tests verify that the embedding generator is invoked only when appropriate, that the repository stores embeddings correctly, and that preprocessing functions are called for valid documents. Additionally, the tests confirm that centroid recomputation is triggered after document processing.

Benefit: These tests validate the reliability of the document processing pipeline, ensuring that embeddings are generated only when necessary and that the system handles various operational scenarios correctly.

C. Similarity Matching Tests

Objective: This test evaluates the similarity matching component responsible for ranking document classes based on cosine similarity between query embeddings and class centroid embeddings.

Test Scenario: A query embedding is generated and compared against multiple class centroid embeddings. The similarity matcher calculates similarity scores and ranks the classes accordingly. Additional tests include cases where the centroid list is empty and where the query embedding is null.

Outcome Verification: The test confirms that the classes are ranked correctly according to similarity scores. It also verifies that the system throws appropriate exceptions when invalid input values are provided and returns an empty result when no centroids are available.

Benefit: These tests ensure that the similarity ranking algorithm correctly identifies the most relevant document categories during query processing.

D. Intent Embedding Generation Tests

Objective: This test verifies the functionality of the intent embedding generation service used to convert user queries into embedding vectors.

Test Scenario: A user query is provided as input to the embedding service, which calls the embedding generator to produce a vector representation of the query. Additional test cases include scenarios where the query input is null or empty, as well as cases where the embedding generator throws an exception.

Outcome Verification: The test confirms that the embedding generator is invoked correctly and that the generated embedding is returned by the service. It also verifies that appropriate exceptions are thrown for invalid input values and that errors from the embedding generator are propagated correctly.

Benefit: These tests ensure that user queries are converted into embeddings reliably, which is essential for accurate semantic similarity comparison.

E. Result Formatting Tests

Objective: This test evaluates the functionality of the result formatting component responsible for displaying classification results to the user.

Test Scenario: The system receives ranked classification results containing class names and similarity scores. The formatter prints the best matching class along with similarity scores for top 3 classes. Another test scenario evaluates the behaviour when no classification results are available.

Outcome Verification: The test verifies that the output contains the correct class names and similarity scores when results are present. It also confirms that an appropriate message is displayed when no matching classes are found.

Benefit: These tests ensure that the classification results presented to users are clear, accurate, and properly formatted.

VII. DISCUSSION

The experimental results demonstrate that embedding-based semantic retrieval combined with class centroid precomputation provides a practical approach for document classification within the evaluated dataset and use cases. The two-stage retrieval pipeline effectively reduces the search space by first identifying the most relevant class using centroid similarity, followed by fine-grained document-level ranking. This design enables efficient retrieval while maintaining meaningful semantic matching.

The observed classification accuracy of approximately 70–75% indicates that the system performs reliably for semantically clear and context-rich queries. As shown in the results section, document-level similarity scores are consistently higher than class-level similarity scores, confirming the effectiveness of the two-stage retrieval mechanism. The centroid-based filtering step successfully narrows down candidate classes, while the second stage refines the results by identifying the most relevant document within the selected category.

A key strength of the system is its modular architecture. The use of well-defined interfaces such as IEmbeddingGenerator, IDocumentPreprocessor, and ISimilarityMetric enables flexibility and extensibility. Components such as the embedding model or preprocessing pipeline can be replaced without affecting the overall system design. This modularity supports future adaptations, including integration of alternative embedding models or different storage solutions.

The system also demonstrates robustness in handling heterogeneous document formats. By integrating OCR through Tesseract, the pipeline is capable of extracting textual content from image-based documents, allowing them to be processed alongside structured formats such as PDF and DOCX. However, the quality of OCR-extracted text can influence embedding quality, which may impact classification performance in certain cases.

Despite these strengths, several limitations are observed. The system shows reduced accuracy for short, ambiguous, or noisy queries, where insufficient semantic context leads to lower similarity scores and less confident classification. Additionally, semantically overlapping categories such as Environment and ClimateChange introduce ambiguity, which can affect classification precision.

From a system design perspective, the current implementation relies on storing embeddings as serialized vectors in an Azure SQL database and performing cosine similarity computations at the application level. While this approach is effective for the current dataset size, it may introduce performance limitations as the number of documents increases. The absence of a dedicated vector index can lead to increased retrieval latency in large-scale deployments.

Future work can address these limitations by incorporating query expansion techniques, contextual disambiguation methods, and improved handling of ambiguous inputs. Additionally, integrating a vector database or indexing mechanism could significantly enhance scalability and retrieval performance. Potential

enhancements also include the integration of Retrieval-Augmented Generation (RAG) to enable natural language responses based on retrieved documents, as well as the development of a web-based interface or API for real-world deployment.

Overall, the results indicate that the combination of semantic embeddings, cosine similarity, and centroid-based classification provides an effective balance between computational efficiency and semantic accuracy. The system demonstrates practical applicability for document classification tasks while highlighting opportunities for further optimization and scalability improvements.

VIII. CONCLUSION

This paper addressed the challenge of efficient document classification and retrieval in large and diverse datasets, where traditional keyword-based approaches are limited in capturing semantic meaning. To solve this, an AI-based system using semantic embeddings and a two-stage similarity retrieval mechanism was proposed. The system successfully processes multiple document formats, including scanned documents through OCR, and performs semantic comparison using vector representations.

The results show that the proposed approach improves classification and retrieval by capturing contextual relationships between queries and documents. The two-stage retrieval strategy enhances efficiency by reducing the search space while maintaining accurate document-level matching. These findings demonstrate the practical value of embedding-based methods for real-world document management systems.

However, the system has some limitations, particularly in handling ambiguous or low-information queries and overlapping categories. Although the introduction of a similarity threshold improves reliability, further refinement is needed to handle such cases more effectively.

Future work can focus on improving dataset structuring, introducing adaptive threshold mechanisms, and integrating more advanced embedding models. Additionally, extending the system to support real-time deployment and larger-scale datasets can further enhance its applicability.

IX. REFERENCES

[1] M. Ikonomakis, S. Kotsiantis, and V. Tampakas, *"Text Classification Using Machine Learning Techniques,"* WSEAS Transactions on Computers, vol. 4, no. 8, pp. 966–974, 2005.

[2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, *"Efficient Estimation of Word Representations in Vector Space,"* in Proc. 1st Int. Conf. on Learning Representations (ICLR), Scottsdale, AZ, USA, May 2013. arXiv:1301.3781.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, *"Attention Is All You Need,"* in Advances in Neural Information Processing Systems (NIPS), vol. 30, pp. 5998–6008, 2017. arXiv:1706.03762.

[4] N. Reimers and I. Gurevych, *"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,"* in Proc. EMNLP-IJCNLP 2019, pp. 3982–3992. arXiv:1908.10084.

[5] T. Gao, X. Yao, and D. Chen, *"SimCSE: Simple Contrastive Learning of Sentence Embeddings,"* in Proc. EMNLP 2021, pp. 6894–6910. DOI: 10.18653/v1/2021.emnlp-main.552. arXiv:2104.08821.

[6] A. Singhal, *"Modern Information Retrieval: A Brief Overview,"* IEEE Data Engineering Bulletin, vol. 24, no. 4, pp. 35–43, 2001.

[7] K. Taunk, S. De, S. Verma, and A. Swetapadma, *"A Brief Review of Nearest Neighbor Algorithm for Learning and Classification,"* in Proc. 2019 Int. Conf. on Intelligent Computing and Control Systems (ICCS), IEEE, pp. 1255–1260. DOI: 10.1109/ICCS45141.2019.9065747.

[8] R. W. Smith, *"An Overview of the Tesseract OCR Engine,"* in Proc. IEEE Ninth Int. Conf. Document Analysis and Recognition (ICDAR 2007), vol. 2, pp. 629–633. DOI: 10.1109/ICDAR.2007.4376991.

[9] OpenAI, *"Embeddings,"* OpenAI Platform Documentation, 2024. [Online]. Available: https://platform.openai.com/docs/guides/embeddings